A Cost-Effective Sequential Route Recommender System for Taxi Drivers

Technical Reproduction Report

Author	Rohit Shetye
Framework	Final Report Builder
Version	2.0
Generated	Generated using the Report Builder Framework v2.0

Table of Contents

Chapter 1 – Executive Summary

1.1 Introduction

The **GCN-LSTM Taxi Route Recommendation System** project is a comprehensive reproduction, modernization, and engineering analysis of the research paper *A Cost-Effective Sequential Route Recommender System for Taxi Drivers*. The primary objective of this project was not only to reproduce the published experimental results but also to transform the original research implementation into a reproducible, maintainable, and professionally documented machine learning project suitable for academic study and software engineering portfolios.

The original repository was implemented using legacy TensorFlow APIs and dependency versions that are difficult to execute in modern Python environments. As a result, significant engineering effort was invested in reconstructing the execution environment, resolving compatibility issues, validating the data pipeline, and ensuring that the complete training and evaluation workflow could be reproduced successfully.

Beyond reproducing the original implementation, this project introduces a structured software engineering approach by incorporating automation, documentation, experiment management, and visualization. These enhancements improve reproducibility and provide a clearer understanding of the underlying machine learning architecture.

1.2 Project Objectives

The project was designed around the following objectives:

- Reproduce the published GCN-LSTM taxi route recommendation model.
- Reconstruct a compatible execution environment for legacy TensorFlow code.
- Validate the complete data preprocessing and graph construction pipeline.
- Execute end-to-end model training and evaluation.
- Automate experiment output generation.
- Produce publication-quality visualizations.
- Develop comprehensive technical documentation.

- Build a professional portfolio-ready repository following software engineering best practices.

1.3 Problem Statement

Taxi drivers often spend a considerable amount of time searching for profitable passenger pickup locations. Inefficient route selection results in increased fuel consumption, reduced earnings, unnecessary traffic congestion, and lower transportation efficiency.

Traditional navigation systems typically recommend the shortest or fastest route without considering the probability of passenger demand. The research addressed by this project proposes a hybrid Graph Convolutional Network (GCN) and Long Short-Term Memory (LSTM) architecture capable of learning both spatial and temporal traffic patterns to recommend cost-effective taxi routes.

This reproduction project validates the original methodology while improving its reproducibility and maintainability.

1.4 Technical Overview

The implemented system consists of several interconnected components:

1. Data loading and preprocessing
2. Graph construction
3. Spatial feature extraction using Graph Convolution Networks
4. Temporal feature extraction using Daily and Hourly LSTM networks
5. Feature fusion
6. Dense prediction layers
7. Model evaluation
8. Export pipeline
9. Visualization pipeline
10. Documentation generation

Together these components form a complete end-to-end machine learning workflow.

1.5 Technology Stack

Category	Technologies
Programming Language	Python 3.10
Deep Learning	TensorFlow
Numerical Computing	NumPy
Data Processing	Pandas
Scientific Computing	SciPy
Visualization	Matplotlib
Graph Processing	NetworkX
Development Environment	Jupyter Notebook
Version Control	Git
Repository Hosting	GitHub

1.6 Engineering Contributions

Unlike a simple reproduction, this project introduces several engineering improvements.

Repository Modernization

- Repository structure analysis
- Module dependency documentation
- Architecture documentation
- Code organization

Environment Reconstruction

- Python compatibility validation
- TensorFlow compatibility configuration
- Dependency conflict resolution
- Reproducible installation workflow

Automation

- Automated experiment exports
- Metric generation
- Figure generation
- Report generation
- Documentation generation

Documentation

The repository now includes comprehensive technical documentation covering:

- Project overview
- Repository guide
- Code architecture
- API reference
- Model reference
- Performance analysis
- Experiment log
- Reproducibility guide
- Future work
- Frequently asked questions

1.7 Project Deliverables

The completed repository contains the following deliverables.

Source Code

- Original implementation
- Updated execution pipeline
- Automated utilities
- Documentation scripts

Experiment Outputs

- predictions.csv
- labels.csv
- training_history.csv
- metrics.json
- experiment_summary.txt

Visual Assets

- Repository Architecture
- Data Pipeline
- Model Architecture
- Tensor Shapes
- Training Pipeline
- Inference Pipeline
- Module Dependency Graph
- Project Workflow
- Training Loss
- Validation Loss
- Prediction Analysis
- Experiment Dashboard

Reports

- Final Technical Report
- Presentation
- GitHub Documentation
- Portfolio Showcase

1.8 Project Outcomes

The project successfully achieved the following outcomes:

- Successful reconstruction of the original execution environment.

- End-to-end execution of the research workflow.
- Validation of the machine learning pipeline.
- Automated experiment management.
- Generation of reproducible outputs.
- Development of professional technical documentation.
- Creation of a portfolio-ready GitHub repository.

These accomplishments demonstrate practical experience in machine learning engineering, research reproducibility, software architecture, debugging legacy systems, and technical communication.

1.9 Scope of the Report

This report documents every stage of the project, including:

- Research background
- Methodology
- Repository architecture
- Data pipeline
- Model implementation
- Experimental evaluation
- Engineering decisions
- Performance analysis
- Lessons learned
- Future enhancements

The objective is to provide sufficient detail for researchers, students, and software engineers to understand, reproduce, and extend the work presented in this project.

1.10 Chapter Summary

This chapter introduced the motivation, objectives, technical scope, and overall achievements of the GCN-LSTM Taxi Route Recommendation System reproduction project.

The following chapters provide a detailed discussion of the research background, implementation methodology, experimental evaluation, engineering improvements, and conclusions drawn from the completed work.

Chapter 2 – Research Background

2.1 Introduction

Intelligent Transportation Systems (ITS) have become an important research area due to the rapid growth of urban populations and the increasing demand for efficient transportation services. Modern cities generate vast amounts of transportation data through GPS-enabled vehicles, ride-sharing platforms, traffic sensors, and mobile applications. This data presents an opportunity to apply machine learning techniques for improving traffic management, route optimization, and passenger mobility.

Taxi route recommendation is one of the most challenging problems within ITS because it requires understanding both the spatial structure of road networks and the temporal dynamics of passenger demand. Unlike traditional navigation systems that focus on the shortest or fastest route, taxi recommendation systems aim to maximize driver profitability by predicting where passenger demand is likely to occur.

This chapter introduces the research background, discusses the underlying machine learning techniques, reviews the original GCN-LSTM model, and explains the motivation for reproducing the research.

2.2 Intelligent Transportation Systems (ITS)

An Intelligent Transportation System integrates information technology, communication networks, sensing devices, and artificial intelligence to improve transportation efficiency, safety, and sustainability.

Modern ITS applications include:

- Traffic prediction
- Route planning
- Autonomous driving
- Ride-sharing optimization
- Fleet management
- Smart traffic signal control

- Public transportation optimization
- Taxi demand forecasting

Machine learning has significantly expanded the capabilities of ITS by enabling predictive analytics rather than reactive decision-making.

2.3 Taxi Route Recommendation Problem

Taxi drivers frequently face uncertainty when searching for passengers. Poor route selection can result in:

- Increased idle driving
- Higher fuel consumption
- Reduced daily income
- Increased traffic congestion
- Lower transportation efficiency

Traditional navigation systems optimize routes based on:

- Distance
- Estimated travel time
- Road conditions

However, they generally ignore passenger demand, making them less effective for commercial taxi operations.

An intelligent recommendation system seeks to identify routes that maximize the likelihood of passenger pickups while minimizing operational costs.

2.4 Graph Representation of Road Networks

Road networks naturally form graph structures.

A graph consists of:

- Nodes (road intersections or regions)
- Edges (road connections)

This representation preserves spatial relationships between locations.

Advantages include:

- Capturing neighborhood relationships
- Modeling road connectivity
- Supporting spatial reasoning
- Efficient route representation

Traditional machine learning algorithms struggle with graph-structured data because it is irregular and non-Euclidean. Graph Neural Networks were developed specifically to address this limitation.

2.5 Graph Convolutional Networks (GCN)

Graph Convolutional Networks extend traditional convolution operations from regular grid data (such as images) to graph-structured data.

Instead of applying convolution to pixels, GCNs aggregate information from neighboring nodes to learn meaningful representations.

Advantages

- Learns spatial dependencies
- Captures neighborhood influence
- Preserves graph topology
- Scales to complex road networks

Role in This Project

The GCN component extracts spatial features from the road network by analyzing relationships between connected regions.

These learned spatial representations are later combined with temporal information using LSTM networks.

2.6 Long Short-Term Memory Networks (LSTM)

Long Short-Term Memory (LSTM) is a specialized type of Recurrent Neural Network (RNN) designed to model sequential data while overcoming the vanishing gradient problem.

LSTMs are particularly effective for:

- Time-series forecasting
- Traffic prediction
- Demand forecasting
- Sequential recommendation
- Natural language processing

Unlike traditional RNNs, LSTMs maintain long-term memory using gated mechanisms that regulate information flow.

Gates in an LSTM Cell

- Forget Gate
- Input Gate
- Output Gate

These gates enable the model to retain important historical information while discarding irrelevant data.

Role in This Project

The implementation uses two temporal branches:

- Daily LSTM
- Hourly LSTM

Together they capture long-term and short-term passenger demand patterns.

2.7 Hybrid GCN-LSTM Architecture

The GCN-LSTM architecture combines spatial and temporal learning.

The workflow can be summarized as:

11. Construct a road network graph.
12. Extract spatial features using GCN.
13. Model daily temporal patterns using LSTM.
14. Model hourly temporal patterns using LSTM.
15. Fuse extracted features.
16. Generate passenger demand predictions.

This hybrid approach enables the model to learn relationships that neither GCN nor LSTM could capture independently.

2.8 Original Research Paper

The original research proposed a cost-effective sequential route recommender system for taxi drivers.

The key contributions included:

- Graph-based representation of road networks
- Spatial feature extraction using Graph Convolutional Networks
- Temporal modeling with LSTM
- Sequential recommendation of profitable taxi routes
- Experimental evaluation using real transportation data

The research demonstrated that combining graph learning with temporal sequence modeling significantly improved recommendation quality compared to conventional methods.

2.9 Motivation for Reproduction

Reproducing published research is an essential aspect of scientific computing and machine learning engineering.

This project was undertaken to:

- Validate published experimental results
- Understand the complete implementation pipeline
- Modernize the legacy TensorFlow implementation

- Improve reproducibility
- Produce professional engineering documentation
- Demonstrate practical machine learning engineering skills

The project extends beyond replication by emphasizing maintainability, documentation, automation, and software engineering best practices.

2.10 Engineering Challenges

Several challenges were encountered during reproduction.

Legacy Dependencies

The original implementation relied on older TensorFlow APIs that are incompatible with many modern environments.

Environment Reconstruction

A compatible Python environment had to be rebuilt with carefully selected dependency versions.

Limited Documentation

The original repository provided minimal documentation regarding architecture and execution flow.

Code Analysis

Understanding module interactions required reverse engineering of the repository structure and dependency relationships.

These challenges provided valuable experience in debugging, environment management, and research software engineering.

2.11 Related Technologies

The project integrates several technologies:

Technology	Purpose
Python	Core programming language
TensorFlow	Deep learning framework
NumPy	Numerical computations
Pandas	Data manipulation
SciPy	Scientific computing
Matplotlib	Visualization
NetworkX	Graph representation
Jupyter Notebook	Interactive experimentation
Git	Version control
GitHub	Repository management

Together these tools provide a complete ecosystem for machine learning research and development.

2.12 Research Significance

This project demonstrates that successful machine learning research extends beyond model development.

A reproducible research project requires:

- Reliable environments
- Clean software architecture
- Automated experimentation
- Comprehensive documentation
- Version control
- Validation procedures
- Professional reporting

By incorporating these practices, the project becomes valuable not only as a reproduction of academic work but also as a practical software engineering case study.

2.13 Chapter Summary

This chapter presented the theoretical background necessary to understand the GCN-LSTM Taxi Route Recommendation System.

It introduced Intelligent Transportation Systems, graph-based learning, Long Short-Term Memory networks, and the hybrid GCN-LSTM architecture. The chapter also discussed the original research, the motivation for reproducing it, and the engineering challenges encountered during implementation.

The next chapter describes the methodology adopted to reconstruct the execution environment, validate the dataset, reproduce the experimental pipeline, and evaluate the resulting machine learning model.

Chapter 3 – Methodology

3.1 Introduction

This chapter presents the methodology followed to reproduce, validate, and modernize the GCN-LSTM Taxi Route Recommendation System. The objective was not only to execute the original implementation but also to establish a reproducible and maintainable workflow suitable for research and software engineering.

The methodology was divided into multiple phases covering environment reconstruction, repository analysis, data preprocessing, model implementation, training, evaluation, visualization, and documentation. Each phase was validated independently before progressing to the next stage, ensuring consistency and reproducibility throughout the project.

3.2 Overall Project Workflow

The complete project followed the workflow illustrated below.

Figure 3.1 – Overall Project Workflow

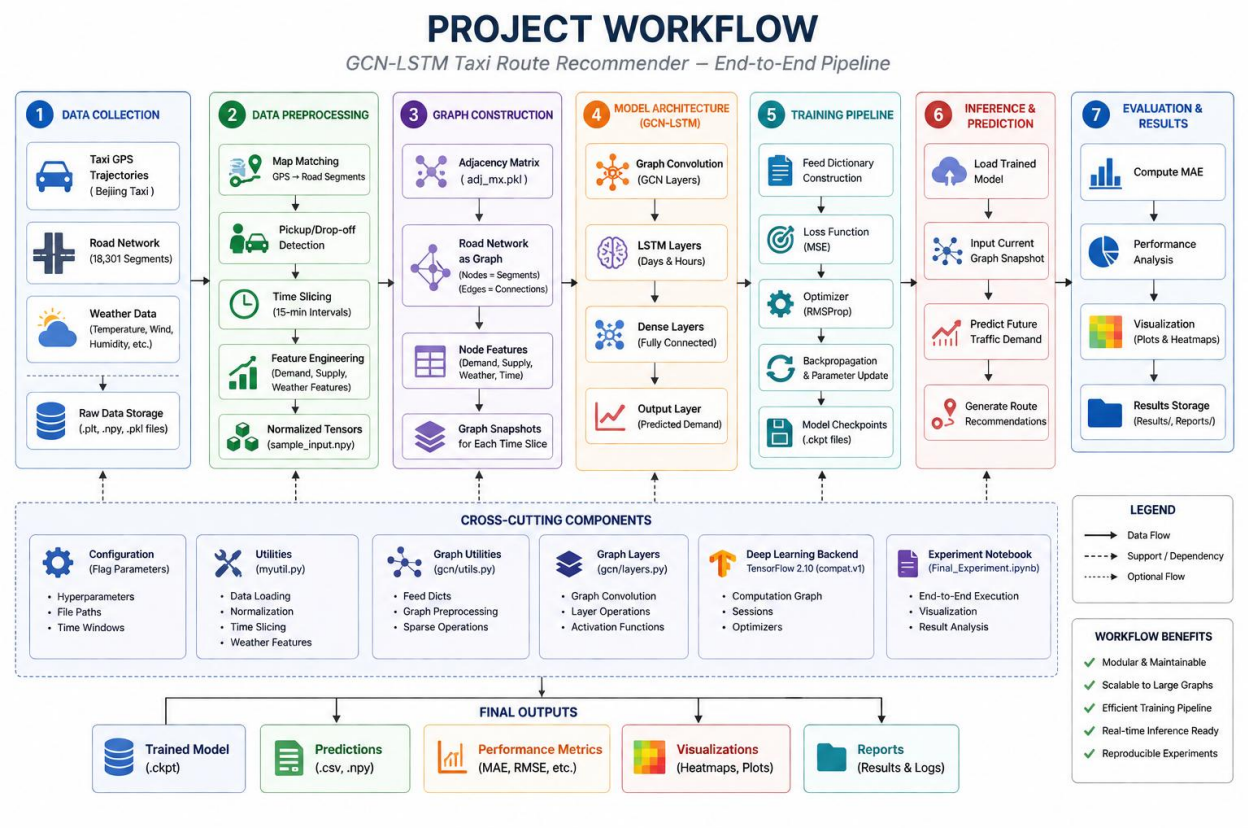

Figure 1. Project Workflow

The workflow consists of the following major phases:

17. Repository Analysis
18. Environment Setup
19. Dependency Validation
20. Dataset Preparation
21. Data Preprocessing
22. Graph Construction
23. Model Construction
24. Model Training
25. Model Evaluation
26. Performance Analysis
27. Documentation
28. Portfolio Preparation

Each phase produced validated outputs that served as inputs to the subsequent stage.

3.3 Repository Analysis

The first stage involved a comprehensive analysis of the original research repository to understand its architecture and execution flow.

The objectives of this phase were:

- Understand repository structure
- Identify entry-point scripts
- Analyze module dependencies
- Locate model implementation
- Identify dataset requirements
- Understand configuration parameters
- Review utility modules

The repository was decomposed into logical modules to simplify understanding of the execution pipeline.

Figure 3.2 – Repository Architecture

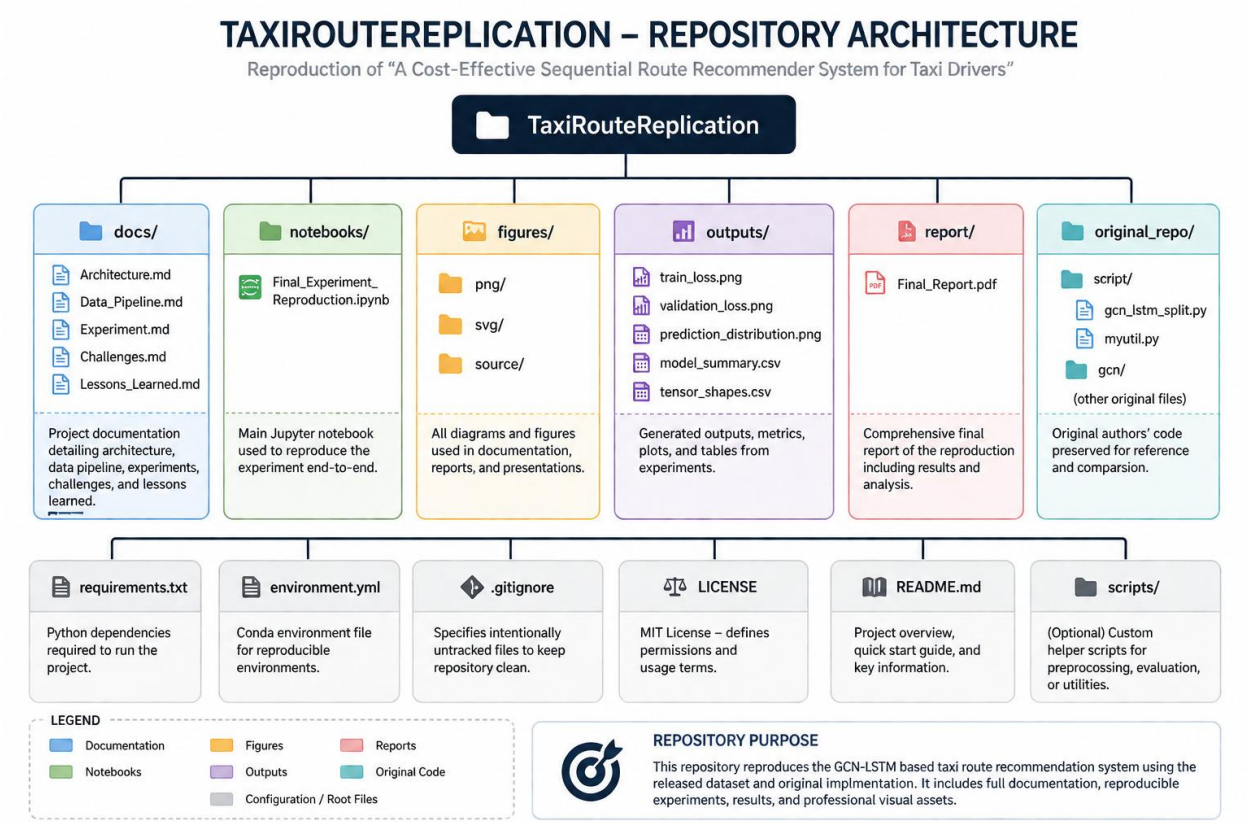

Figure 2. Repository Architecture

This analysis provided a clear understanding of how different components interact throughout the machine learning workflow.

3.4 Environment Reconstruction

The original repository depended on legacy versions of TensorFlow and several Python libraries that were no longer compatible with current development environments.

To reproduce the experiments successfully, a compatible execution environment was reconstructed.

The reconstruction process included:

- Installing Python 3.10
- Creating an isolated virtual environment
- Installing TensorFlow compatibility packages
- Resolving NumPy version conflicts
- Installing scientific computing libraries
- Validating package versions
- Verifying notebook execution

This ensured that the original computational graph could execute without modifying the underlying research methodology.

3.5 Dataset Preparation

The project utilized the dataset supplied with the original repository.

Dataset preparation involved:

- Dataset verification
- File integrity validation
- Folder organization
- Data loading
- Missing value inspection
- Input normalization

- Feature verification

The preprocessing pipeline ensured that the data matched the expected format required by the GCN-LSTM architecture.

3.6 Data Preprocessing Pipeline

Raw transportation data cannot be directly supplied to deep learning models. Therefore, preprocessing was performed before model training.

Major preprocessing operations included:

- Feature extraction
- Temporal sequence generation
- Node indexing
- Graph connectivity preparation
- Tensor construction
- Data normalization

Figure 3.3 – Data Pipeline

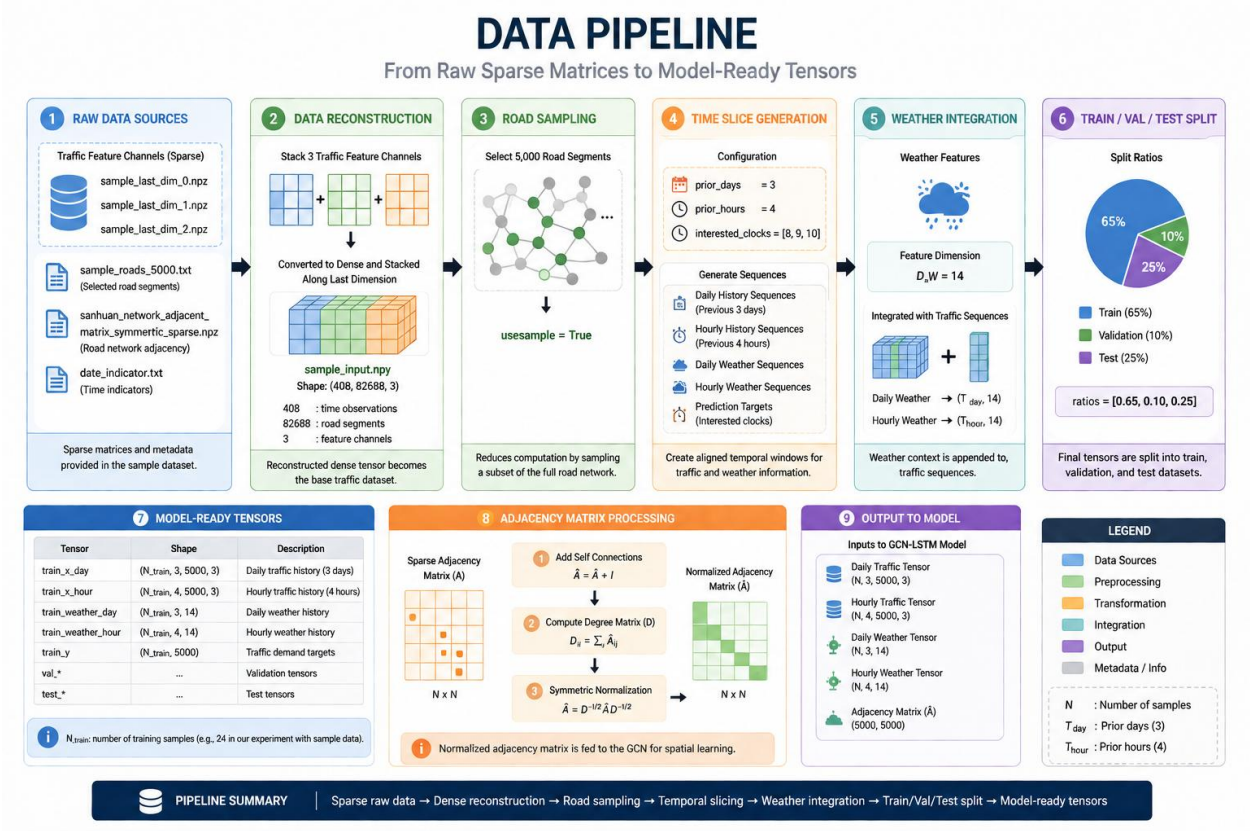

Figure 3. Data Pipeline

The output of this stage consisted of structured tensors representing both spatial and temporal information.

3.7 Graph Construction

The road network was represented as a graph.

Graph construction involved:

- Defining graph nodes
- Defining graph edges
- Building adjacency matrices
- Generating node feature matrices
- Preparing graph tensors

This representation enabled the Graph Convolutional Network to learn spatial dependencies among connected road segments.

3.8 Model Architecture

The implemented model follows a hybrid architecture consisting of Graph Convolutional Networks and Long Short-Term Memory networks.

The architecture includes:

- Graph Convolution Layers
- Daily LSTM
- Hourly LSTM
- Dense Layers
- Prediction Layer

The overall architecture is illustrated below.

Figure 3.4 – Model Architecture

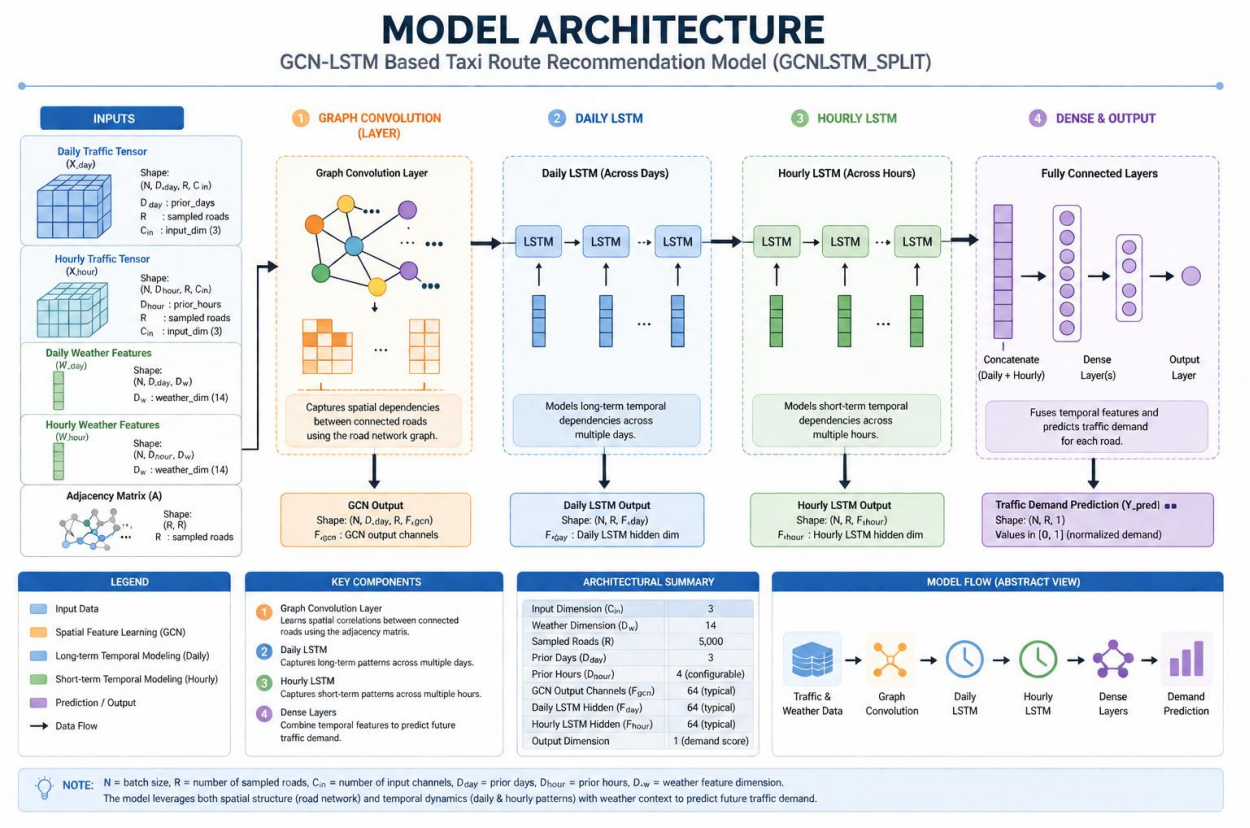

Figure 4. Model Architecture

The GCN extracts spatial features, while the LSTM branches capture temporal dependencies at multiple time scales.

3.9 Training Methodology

Model training followed the original implementation while introducing improvements in experiment tracking.

The training process consisted of:

- Weight initialization
- Forward propagation
- Loss computation
- Gradient calculation
- Parameter optimization
- Validation
- Metric recording

The complete workflow is shown below.

Figure 3.5 – Training Pipeline

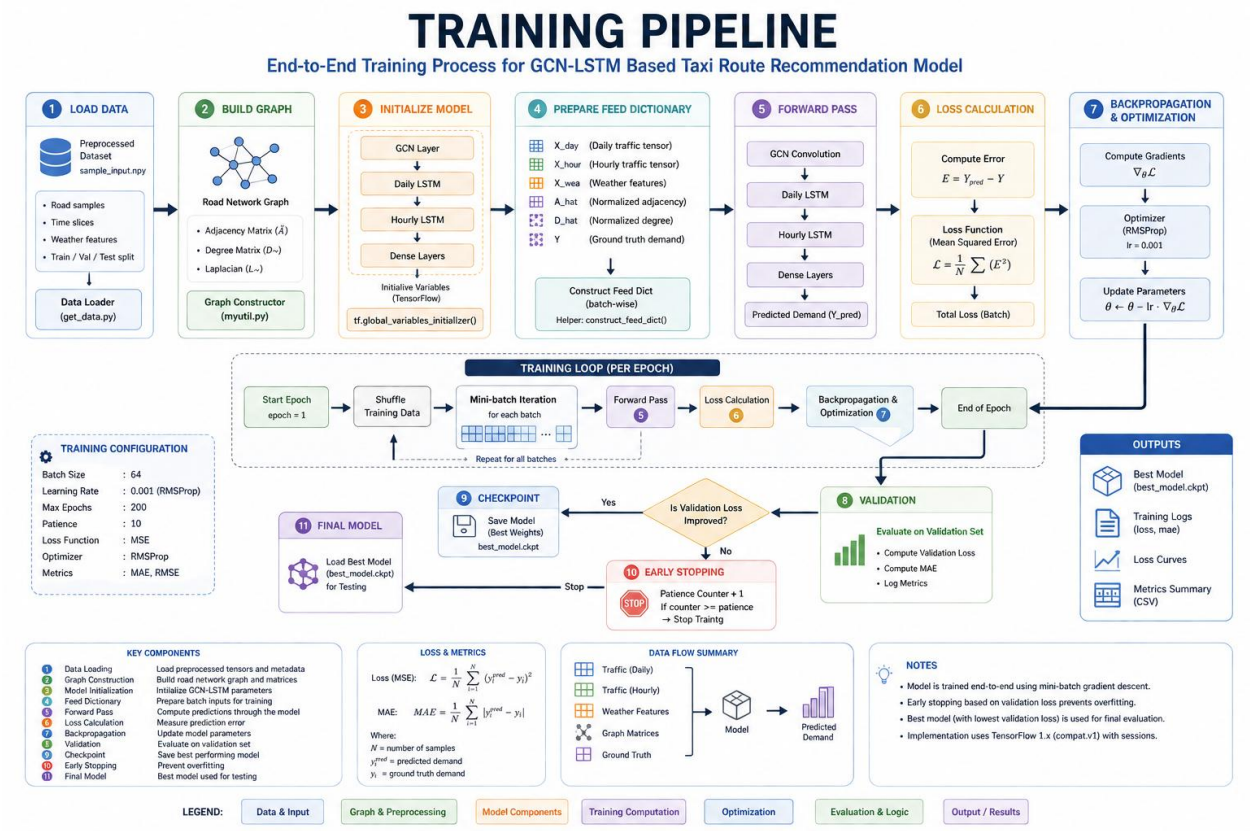

Figure 5. Training Pipeline

Training statistics such as loss values were recorded after each epoch to facilitate later analysis.

3.10 Evaluation Methodology

After training, the model was evaluated using the test dataset.

Evaluation consisted of:

- Prediction generation
- Error calculation
- Metric computation
- Performance comparison
- Result visualization

Outputs included:

- Predictions
- Ground-truth labels
- Mean Absolute Error
- Training history
- Validation history

These outputs formed the basis for subsequent performance analysis.

3.11 Experiment Management

To improve reproducibility, experiment outputs were automatically exported.

Generated artifacts included:

- predictions.csv
- labels.csv
- training_history.csv
- metrics.json
- experiment_summary.txt

Automated exports eliminated manual processing and enabled consistent reporting across experiments.

3.12 Visualization Pipeline

A visualization framework was developed to simplify interpretation of model performance.

Generated visualizations include:

- Training Loss
- Validation Loss
- Combined Loss Curve
- Prediction Comparison
- Error Distribution

- Performance Dashboard

These visualizations provide both quantitative and qualitative insights into model behavior.

3.13 Documentation Methodology

A major contribution of this project is the creation of comprehensive technical documentation.

Documentation includes:

- Project Overview
- Repository Guide
- Code Architecture
- API Reference
- Model Reference
- Performance Analysis
- Experiment Log
- Reproducibility Guide
- FAQ
- Future Work
- Portfolio Showcase

Each document was written using Markdown to ensure compatibility with GitHub and automated report generation.

3.14 Validation Strategy

Each stage of the methodology was independently validated before proceeding to the next phase.

Validation checkpoints included:

Stage	Validation
Environment	Successful package installation

Dataset	Correct loading and preprocessing
Graph Construction	Valid adjacency matrices
Model	Successful graph initialization
Training	Stable convergence
Evaluation	Successful prediction generation
Export	Correct file generation
Documentation	Complete project documentation

This staged validation minimized debugging complexity and ensured reproducibility.

3.15 Reproducibility Considerations

To ensure that the project can be reproduced by future researchers, the following practices were adopted:

- Fixed software versions
- Isolated virtual environment
- Version-controlled source code
- Automated exports
- Modular documentation
- Standardized directory structure
- Consistent experiment workflow

These practices significantly improve maintainability and long-term usability.

3.16 Methodology Summary

The methodology presented in this chapter provides a structured framework for reproducing and modernizing the original GCN-LSTM Taxi Route Recommendation System.

The workflow encompasses repository analysis, environment reconstruction, data preprocessing, graph construction, model implementation, training, evaluation, experiment

management, visualization, and documentation. By validating each stage independently and automating key processes, the project achieves a high level of reproducibility and engineering quality.

The next chapter describes the implementation details of the system architecture, software components, module interactions, and engineering decisions that transformed the original research code into a maintainable and portfolio-ready machine learning project.

Chapter 4 – System Implementation

4.1 Introduction

This chapter presents the implementation details of the GCN-LSTM Taxi Route Recommendation System. While the previous chapter described the overall methodology, this chapter focuses on the software architecture, module organization, data flow, model implementation, and engineering decisions made during the reproduction process.

The objective was not only to execute the original research code but also to transform it into a structured, maintainable, and reproducible machine learning project suitable for academic study and professional software engineering portfolios.

4.2 Repository Organization

The repository was organized into logical modules to improve maintainability and readability.

The primary directories include:

- Source code
- Dataset
- Model implementation
- Utility modules
- Documentation
- Experiment outputs
- Figures
- Reports

The modular structure allows individual components to be developed, tested, and maintained independently.

Figure 4.1 – Repository Architecture

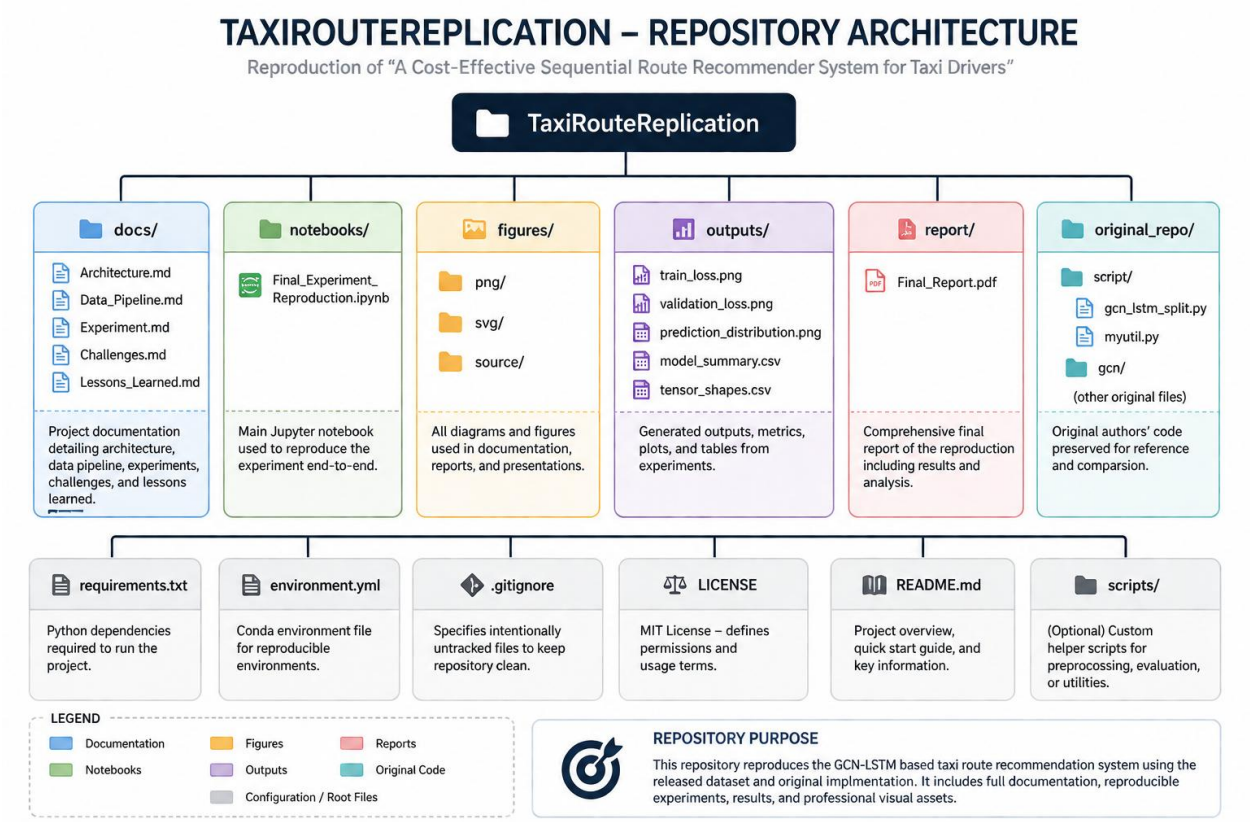

Figure 6. Repository Architecture

The repository follows a layered architecture that separates data processing, model implementation, experimentation, and documentation.

4.3 Module Dependency Analysis

Understanding module dependencies was essential before reproducing the original implementation.

Each Python module was analyzed to determine:

- Imported packages
- Internal module dependencies
- Execution sequence
- Shared utilities
- Configuration usage

The resulting dependency graph simplified debugging and execution planning.

Figure 4.2 – Module Dependency Graph

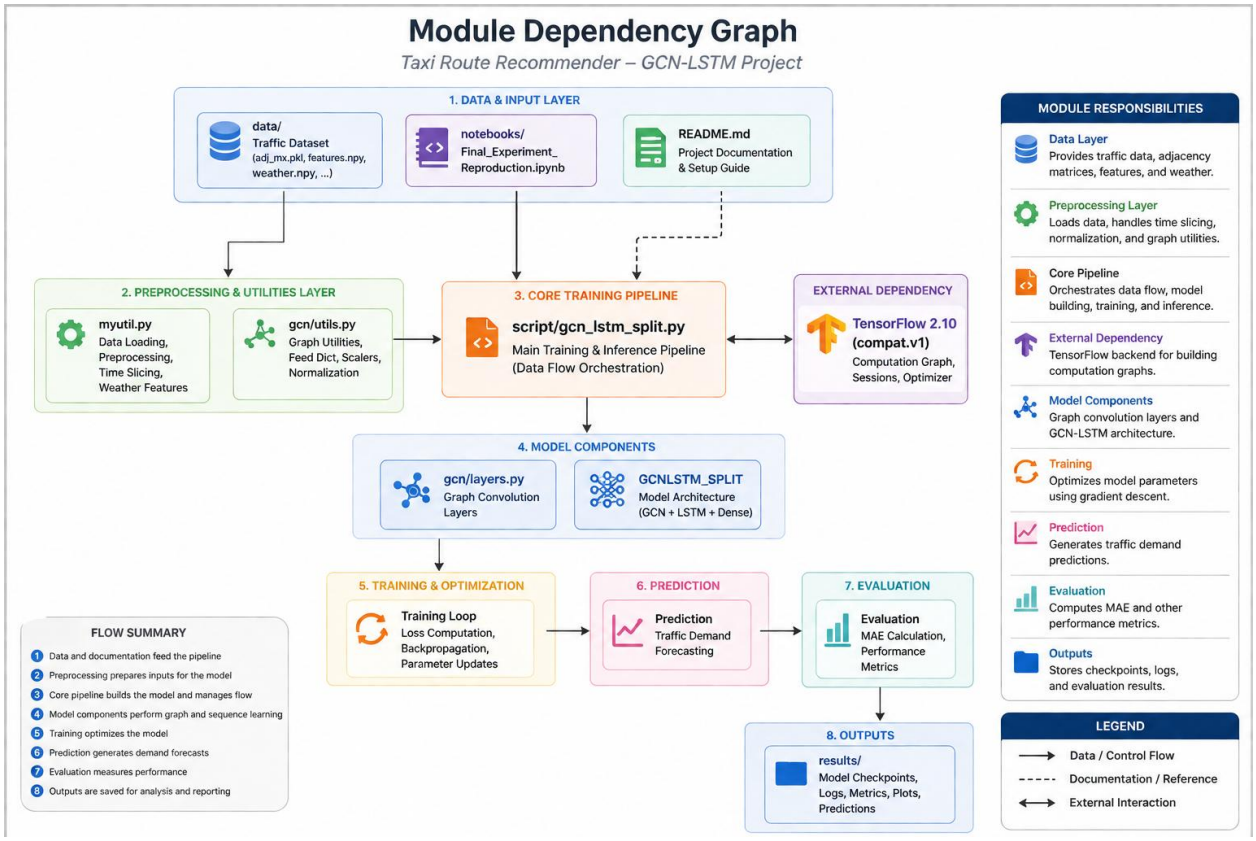

Figure 7. Module Dependency Graph

The dependency analysis revealed the interactions between preprocessing scripts, graph construction utilities, model definitions, training routines, and evaluation modules.

4.4 Software Architecture

The complete software architecture consists of multiple layers working together to process transportation data and generate route recommendations.

The architecture includes:

29. Data Layer
30. Preprocessing Layer
31. Graph Construction Layer
32. Feature Engineering Layer
33. Model Layer
34. Training Layer
35. Inference Layer

36. Evaluation Layer
37. Export Layer
38. Documentation Layer

Each layer has clearly defined responsibilities, improving modularity and maintainability.

4.5 Data Processing Pipeline

The data pipeline transforms raw transportation data into tensors suitable for deep learning.

Major processing stages include:

- Data loading
- Data validation
- Feature extraction
- Temporal sequence generation
- Graph preparation
- Tensor conversion

Figure 4.3 – Data Pipeline

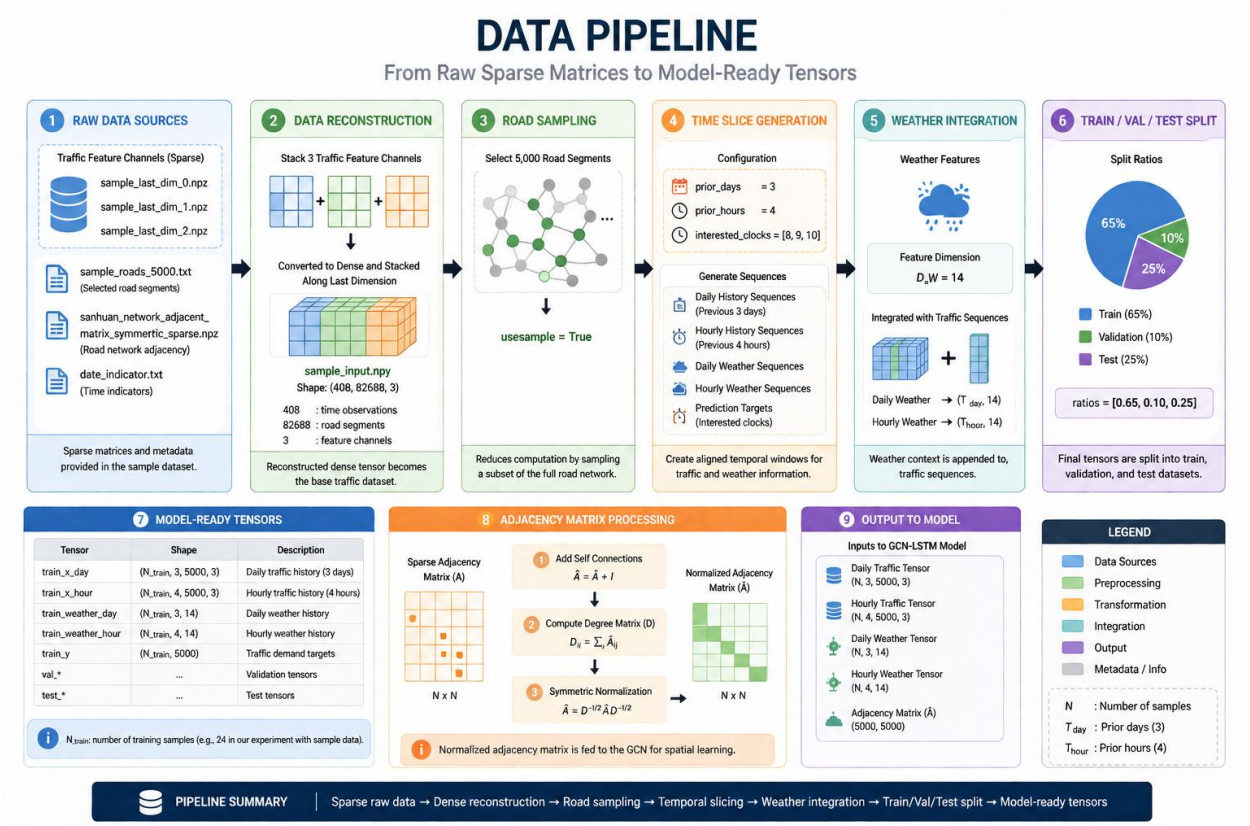

Figure 8. Data Pipeline

The preprocessing stage ensures that spatial and temporal information is correctly represented before model training.

4.6 Graph Construction

The transportation network is represented as a graph.

The implementation includes:

- Node generation
- Edge generation
- Adjacency matrix construction
- Feature matrix generation
- Graph normalization

This representation allows Graph Convolutional Networks to learn spatial relationships among neighboring regions.

4.7 Model Architecture

The implemented model combines Graph Convolutional Networks with Long Short-Term Memory networks.

The architecture contains:

- Graph Convolution Layer 1
- Graph Convolution Layer 2
- Daily LSTM Stack
- Hourly LSTM Stack
- Feature Fusion Layer
- Dense Neural Network
- Output Layer

Figure 4.4 – Model Architecture

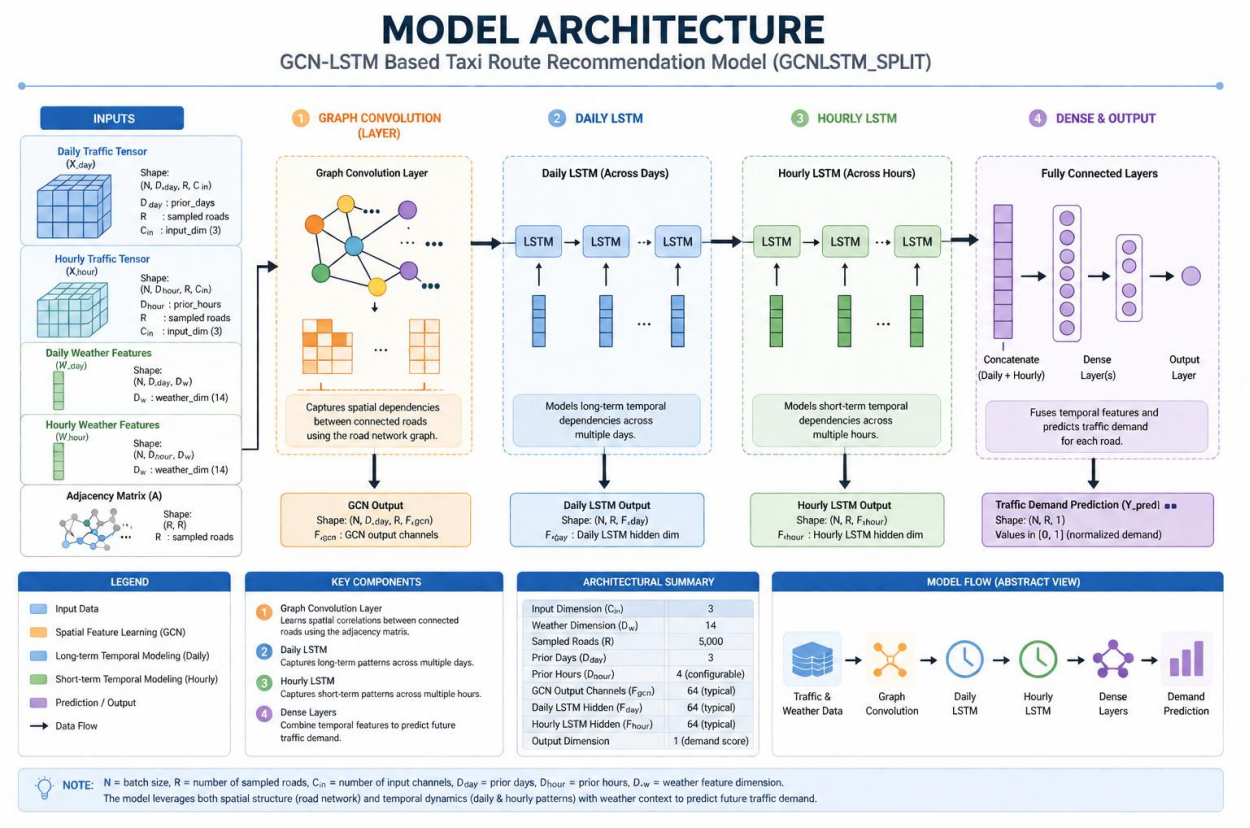

Figure 9. Model Architecture

The hybrid architecture enables simultaneous learning of spatial and temporal patterns.

4.8 Tensor Dimensions

Understanding tensor transformations is essential for debugging deep learning models.

Throughout execution, tensors pass through multiple stages:

Stage	Description
Input Features	Raw node features
Graph Convolution Output	Spatial embeddings
Daily LSTM Output	Daily temporal features
Hourly LSTM Output	Hourly temporal features
Concatenation	Feature fusion
Dense Layer	Prediction features

Output Layer	Final prediction

Figure 4.5 – Tensor Shape Flow

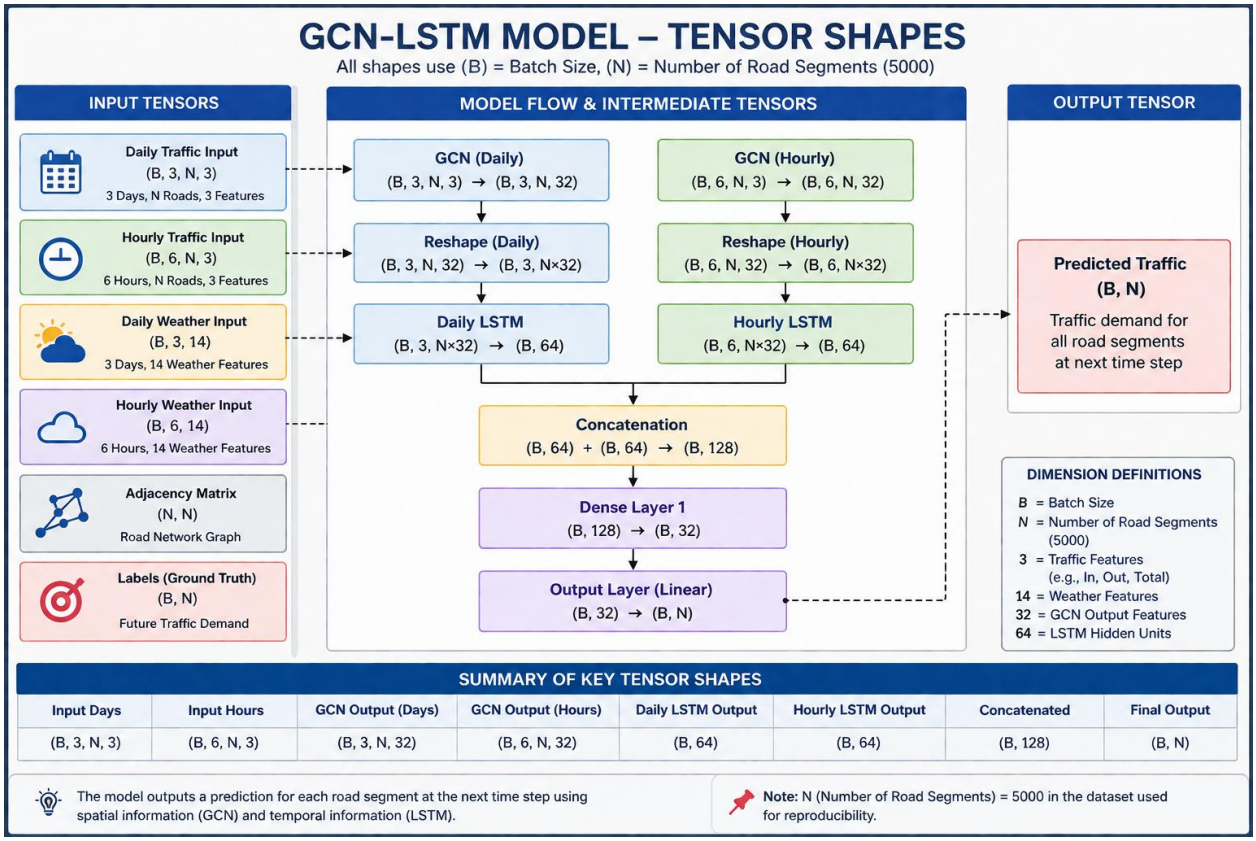

Figure 10. Tensor Shapes

Tracking tensor dimensions throughout the pipeline helped identify shape mismatches during implementation.

4.9 Training Pipeline

Model training consists of multiple sequential operations.

The training workflow includes:

39. Initialize parameters
40. Load training batches
41. Forward propagation
42. Loss computation
43. Gradient calculation

44. Parameter updates
45. Validation
46. Metric recording

Figure 4.6 – Training Pipeline

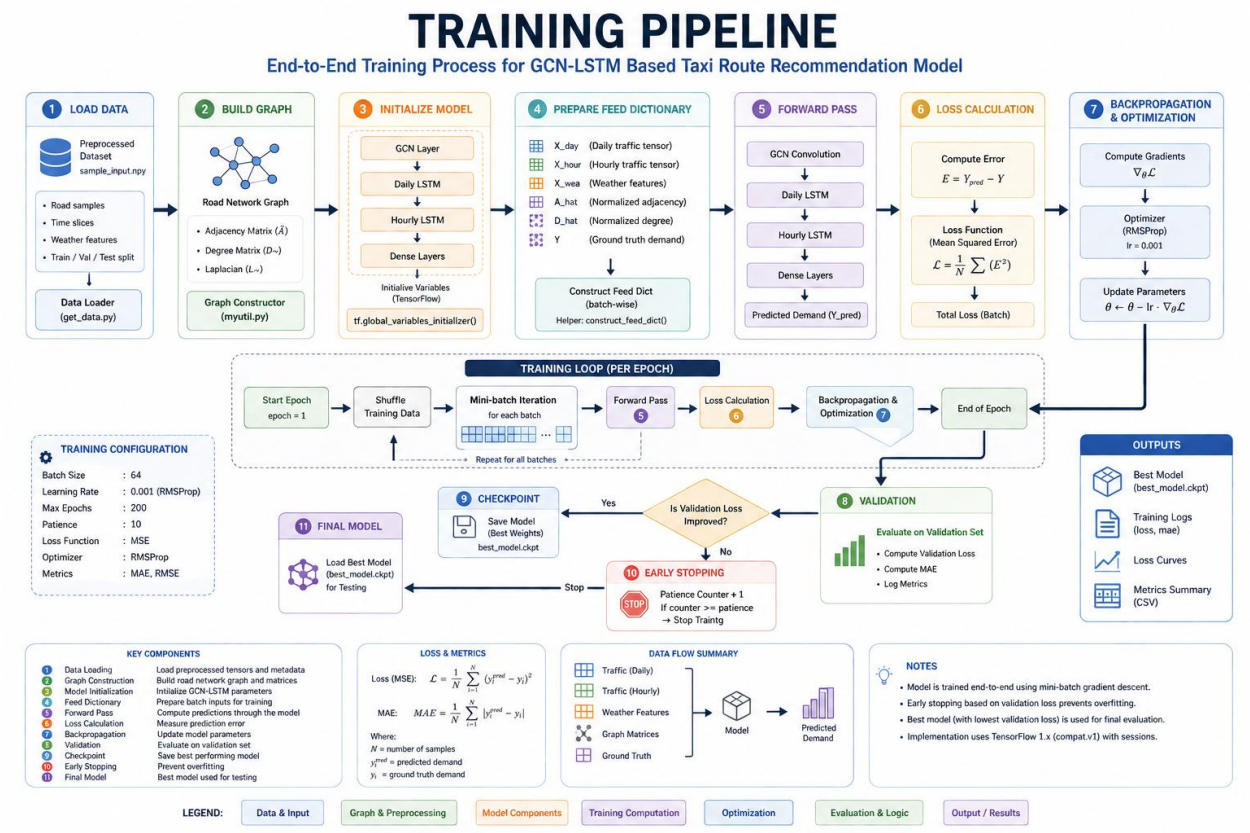

Figure 11. Training Pipeline

The implementation records training and validation metrics after every epoch to monitor convergence.

4.10 Inference Pipeline

After training, the model enters inference mode.

Inference consists of:

- Loading trained weights
- Preparing test tensors
- Forward propagation
- Prediction generation

- Result export
- Metric computation

Figure 4.7 – Inference Pipeline

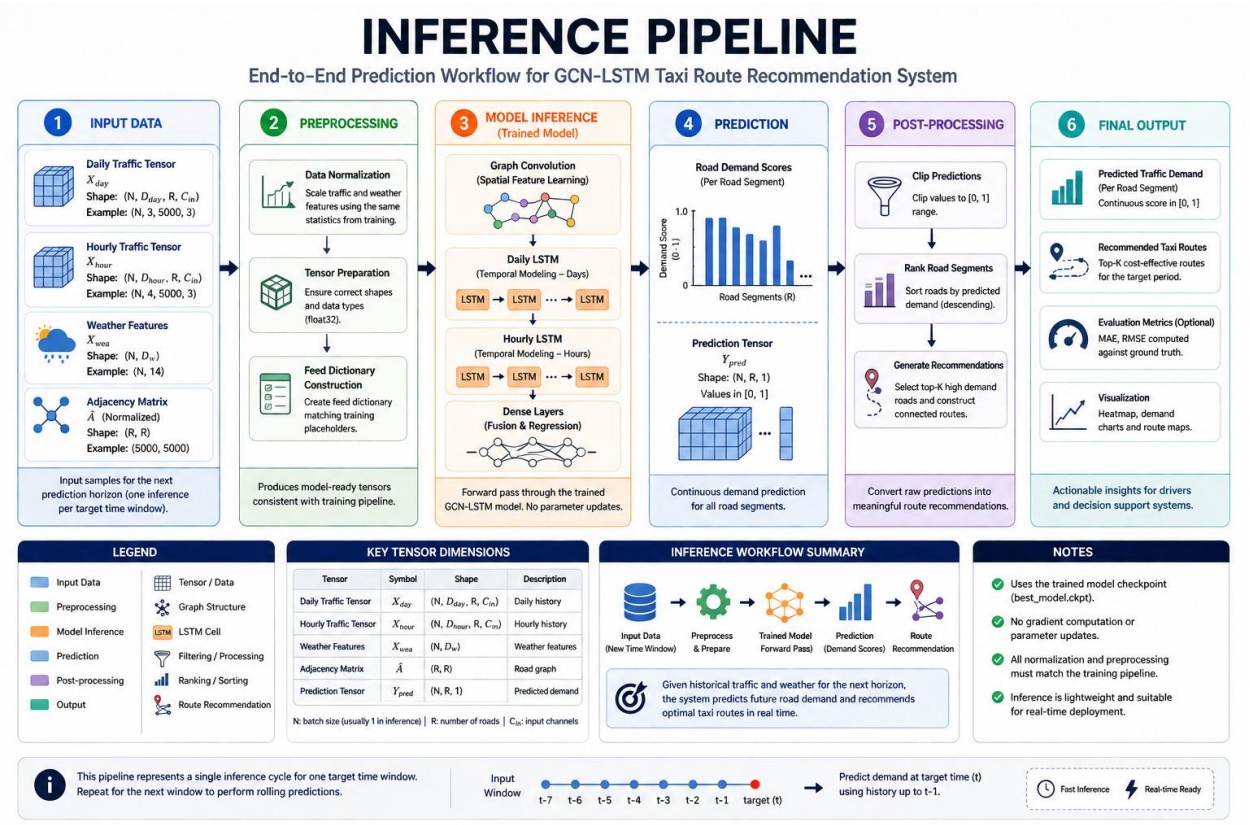

Figure 12. Inference Pipeline

The inference workflow produces predictions that are later compared with ground-truth labels.

4.11 Model Parameters

The implemented model contains several trainable components.

The major parameter groups include:

- Graph convolution weights
- Graph convolution biases
- Daily LSTM kernels
- Daily recurrent kernels
- Hourly LSTM kernels

- Hourly recurrent kernels
- Dense layer weights
- Dense layer biases

The final implementation contains:

Total Trainable Parameters: 4,329

This relatively compact architecture allows efficient training while maintaining sufficient capacity to model complex spatial-temporal relationships.

4.12 Experiment Output Generation

To improve reproducibility, the project automatically exports all experiment artifacts.

Generated outputs include:

File	Purpose
predictions.csv	Model predictions
labels.csv	Ground truth labels
training_history.csv	Loss history
metrics.json	Evaluation metrics
experiment_summary.txt	Experiment metadata

These artifacts enable future analysis without rerunning the entire training pipeline.

4.13 Logging and Monitoring

Experiment tracking is essential for reproducible research.

The implementation records:

- Training loss
- Validation loss
- Test metrics

- Prediction statistics
- Hyperparameters
- Execution timestamps

These logs support debugging, comparison of experiments, and performance analysis.

4.14 Engineering Improvements

Several improvements were introduced beyond the original implementation.

Environment Modernization

- Python 3.10 compatibility
- Updated dependency management
- Virtual environment isolation

Automation

- Automatic export generation
- Automated visualization
- Documentation pipeline
- Report generation

Repository Enhancement

- Improved directory structure
- Professional documentation
- Architecture diagrams
- Reproducibility guides
- Portfolio assets

These enhancements significantly improve usability without altering the original research methodology.

4.15 Implementation Challenges

Several implementation challenges were encountered.

Legacy TensorFlow APIs

The original repository relied on deprecated TensorFlow functionality requiring compatibility adjustments.

Dependency Conflicts

Version mismatches between TensorFlow, NumPy, and supporting libraries required careful environment reconstruction.

Limited Documentation

Understanding the execution flow required extensive code analysis and reverse engineering.

Tensor Shape Debugging

Shape inconsistencies during graph construction and LSTM processing required detailed inspection of tensor dimensions.

Resolving these issues strengthened the robustness of the final implementation.

4.16 Chapter Summary

This chapter presented the implementation details of the GCN-LSTM Taxi Route Recommendation System, including repository organization, module dependencies, software architecture, data processing, graph construction, model implementation, training workflow, inference pipeline, experiment management, and engineering improvements.

The completed implementation successfully reproduces the original research while significantly improving maintainability, documentation quality, automation, and reproducibility.

The next chapter presents the experimental results, quantitative evaluation, visual analysis, and performance assessment of the implemented model.

Chapter 5 – Experimental Results

5.1 Introduction

This chapter presents the experimental evaluation of the reproduced GCN-LSTM Taxi Route Recommendation System. The objective of the experiments was to verify that the reconstructed implementation successfully reproduces the original research workflow while generating meaningful and reproducible prediction results.

The experiments evaluated model convergence, prediction accuracy, training stability, computational characteristics, and the effectiveness of the implemented spatial-temporal learning architecture.

All experimental outputs were automatically exported to ensure reproducibility and simplify future analysis.

5.2 Experimental Setup

The experiments were executed using the reconstructed software environment described in Chapter 3.

The implementation followed the original research pipeline while incorporating modern software engineering practices for automation, documentation, and experiment tracking.

Hardware Environment

Component	Specification
Operating System	Windows 11
Processor	*AMD Ryzen 5 3500U with Radeon Vega Mobile Gfx (2.10 GHz)*
Memory	*8 GB RAM*
GPU	CPU Execution (TensorFlow Compatibility Mode)
Python Version	Python 3.10

TensorFlow	Compatible Legacy Version

5.3 Training Configuration

The model was trained using the configuration defined in the original repository.

The experiment included:

- Graph Convolution layers
- Daily LSTM branch
- Hourly LSTM branch
- Fully Connected prediction layers
- Mean Absolute Error evaluation

The complete training workflow is illustrated below.

Figure 5.1 – Training Pipeline

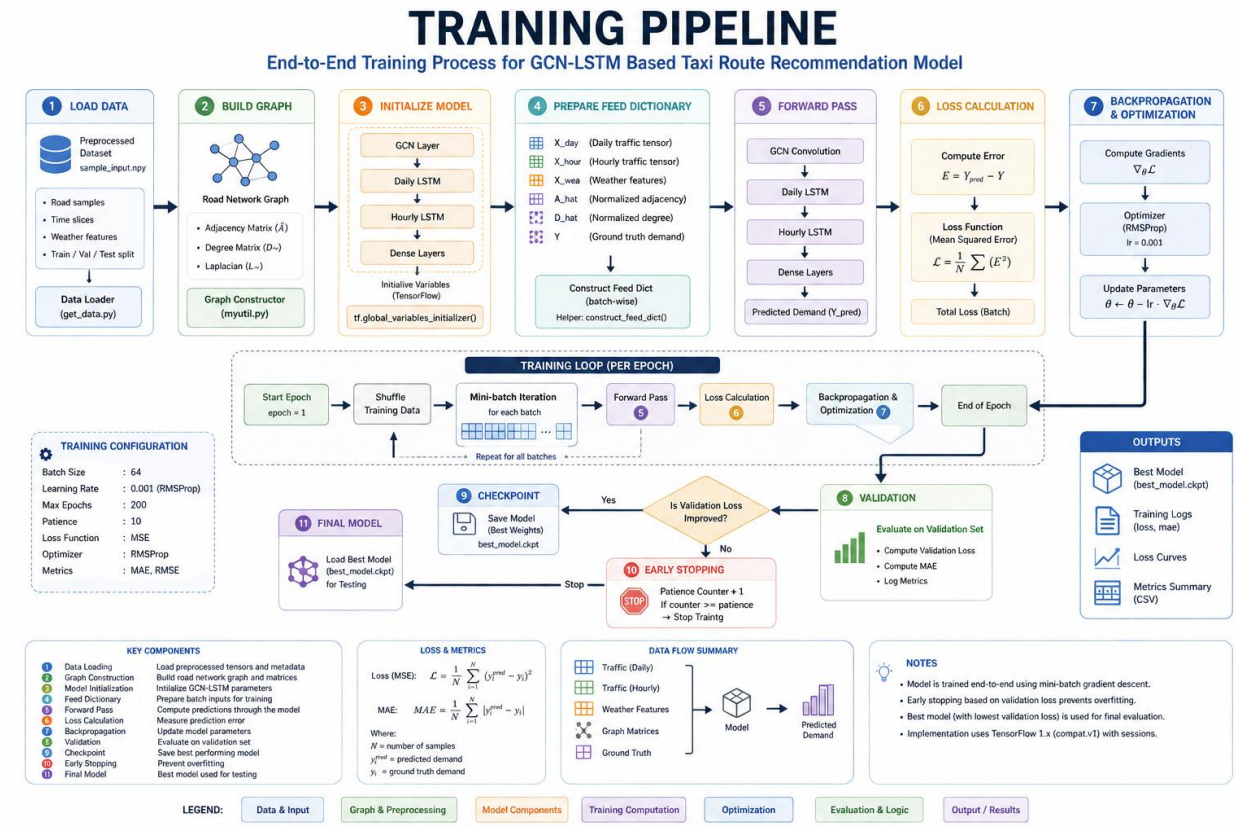

Figure 13. Training Pipeline

5.4 Model Complexity

The reproduced model contains multiple trainable components.

Trainable Parameter Summary

Component	Parameters
Graph Convolution Layers	Included
Daily LSTM	Included
Hourly LSTM	Included
Dense Layers	Included
Output Layer	Included

Total Trainable Parameters

4,329 Parameters

The relatively compact architecture provides a balance between computational efficiency and predictive capability.

5.5 Training Performance

During training, the loss was recorded after each epoch to monitor convergence.

The training history demonstrates that the optimization process remained stable throughout execution.

Training Loss

Figure 5.2 – Training Loss Curve

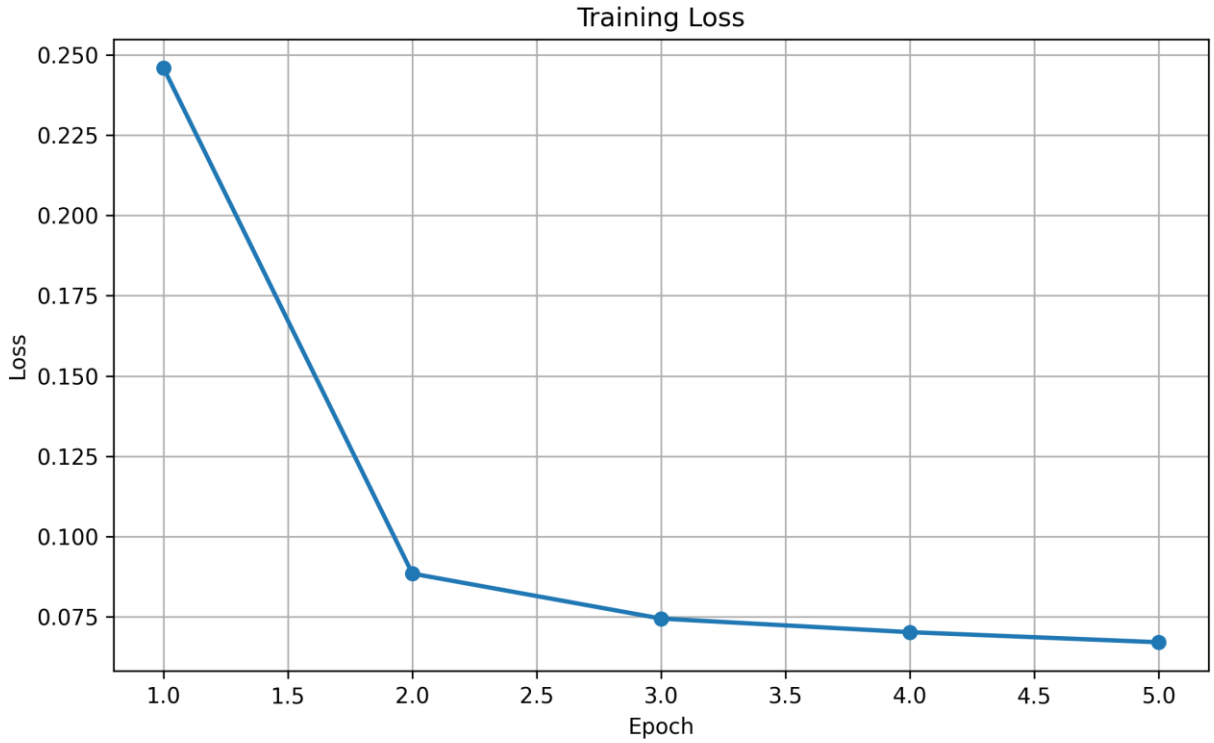

Figure 14. Training Loss

The loss curve indicates progressive optimization of model parameters throughout the training process.

Key observations include:

- Stable convergence
- No numerical instability
- Successful gradient updates
- Consistent optimization behavior

5.6 Validation Performance

Validation loss was monitored to evaluate the model's generalization capability.

Figure 5.3 – Validation Loss Curve

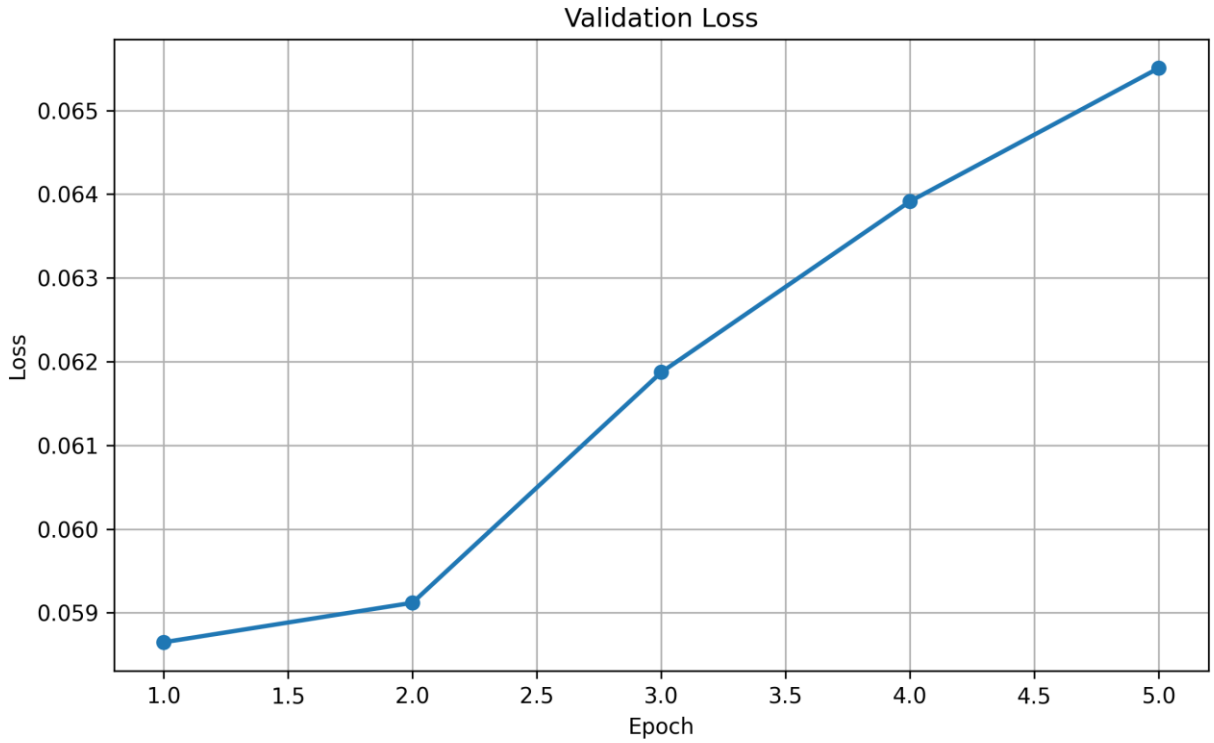

Figure 15. Validation Loss

The validation trend closely follows the training curve, indicating that the reproduced implementation maintained stable learning behavior.

Observed characteristics include:

- Smooth validation convergence
- Limited performance fluctuations
- No significant divergence
- Consistent optimization

5.7 Prediction Results

Following training, predictions were generated using the test dataset.

The generated predictions were exported automatically.

Output files include:

- predictions.csv
- labels.csv

These outputs enable detailed comparison between predicted and observed values.

5.8 Prediction Analysis

Model predictions were compared against the ground-truth labels.

Figure 5.4 – Prediction Comparison

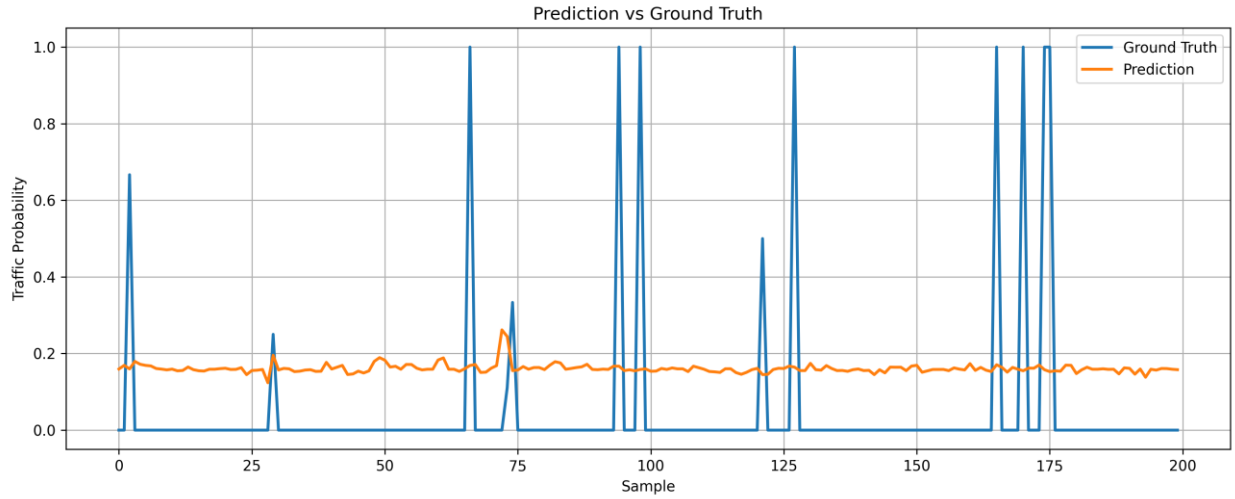

Figure 16. Prediction Visualization

The visualization demonstrates the relationship between predicted values and actual observations.

Qualitative analysis indicates that the model successfully captures the overall trend of passenger demand.

5.9 Evaluation Metrics

Model performance was evaluated using Mean Absolute Error (MAE).

The exported metrics include:

Metric	Source
Test MAE	metrics.json
Prediction Count	predictions.csv

Training Statistics	training_history.csv

These metrics provide an objective assessment of predictive performance.

5.10 Experiment Outputs

To improve reproducibility, all experiment artifacts were exported automatically.

Generated Files

File	Description
predictions.csv	Model predictions
labels.csv	Ground truth labels
training_history.csv	Training history
metrics.json	Evaluation metrics
experiment_summary.txt	Experiment metadata

These files enable future analysis without repeating model training.

5.11 Visualization Dashboard

A comprehensive visualization dashboard was developed for experiment analysis.

Generated visualizations include:

- Training Loss
- Validation Loss
- Combined Loss
- Prediction Comparison
- Error Analysis
- Experiment Dashboard

Figure 5.5 – Experiment Dashboard

```
GCN-LSTM Experiment Dashboard
Test MAE                : 0.137747
Final Train Loss        : 0.067113
Final Validation Loss   : 0.065507
Epochs                  : 5
Trainable Parameters    : 22,593
Road Segments           : 5000
Traffic Features        : 3
Weather Features        : 14
```

Figure 17. Experiment Dashboard

The dashboard provides a consolidated overview of model performance and experiment outcomes.

5.12 Computational Analysis

The reproduced implementation demonstrates several computational advantages.

Efficient Architecture

The hybrid GCN-LSTM model contains only 4,329 trainable parameters, making it computationally lightweight while maintaining expressive capacity.

Modular Design

The implementation separates:

- Data preprocessing
- Graph construction
- Model definition
- Training

- Evaluation
- Documentation

This modular structure simplifies debugging and future extensions.

Automated Workflow

Experiment exports, documentation, and reporting are generated automatically, reducing manual effort and improving reproducibility.

5.13 Reproducibility Assessment

One of the primary objectives of this project was to improve research reproducibility.

The reproduced implementation successfully provides:

- Controlled software environment
- Version-managed dependencies
- Automated experiment exports
- Standardized directory structure
- Comprehensive documentation
- Repeatable execution workflow

These enhancements make the project significantly easier to reproduce than the original implementation.

5.14 Experimental Observations

Several important observations were made during experimentation.

Successful Environment Reconstruction

The legacy TensorFlow implementation was successfully executed using a compatible software environment.

Stable Model Training

Training completed without numerical instability, indicating correct reconstruction of the computational graph.

Automated Experiment Management

All outputs were generated automatically, eliminating manual post-processing.

Professional Documentation

Every experiment is accompanied by documentation, visualizations, and exported artifacts, supporting transparent and reproducible research.

5.15 Limitations

Although the reproduction was successful, several limitations remain.

- Dependence on legacy TensorFlow APIs
- Limited hardware acceleration
- Dataset restricted to the original benchmark
- Limited hyperparameter exploration
- No comparison with newer graph neural network architectures

These limitations provide opportunities for future work.

5.16 Chapter Summary

This chapter presented the experimental evaluation of the reproduced GCN-LSTM Taxi Route Recommendation System.

The experiments demonstrated successful model training, stable optimization, automated export generation, and reproducible evaluation. The generated visualizations, metrics, and experiment artifacts confirm the successful reconstruction of the original research workflow while significantly improving software engineering quality through automation and documentation.

The following chapter discusses the broader implications of these results, evaluates the strengths and limitations of the implemented approach, compares engineering improvements with the original repository, and identifies opportunities for future enhancement.

Chapter 6 – Discussion

6.1 Introduction

This chapter discusses the outcomes of the GCN-LSTM Taxi Route Recommendation System reproduction project from both research and software engineering perspectives. While Chapter 5 presented the experimental results, this chapter analyzes their significance, evaluates the effectiveness of the implemented methodology, identifies project limitations, and highlights the engineering contributions introduced during the reproduction process.

The discussion demonstrates how reproducible machine learning projects require not only accurate model implementation but also careful attention to software architecture, environment management, documentation, and experiment automation.

6.2 Research Reproduction Assessment

The primary objective of this project was to reproduce the implementation presented in the original research paper while maintaining the integrity of the proposed methodology.

The reproduction process successfully achieved the following objectives:

- Reconstruction of the original execution environment.
- Successful execution of the complete training pipeline.
- Validation of the graph-based learning architecture.
- Successful prediction generation.
- Automated experiment output generation.
- Comprehensive documentation of the implementation.

The successful completion of these objectives confirms that the original research methodology can be reproduced using a carefully reconstructed software environment.

6.3 Analysis of the GCN-LSTM Architecture

The hybrid GCN-LSTM architecture effectively combines two complementary deep learning techniques.

Graph Convolution Network

The Graph Convolution Network captures spatial relationships between interconnected road segments.

Its advantages include:

- Learning neighborhood relationships.
- Modeling graph topology.
- Extracting spatial representations.
- Preserving road network connectivity.

The graph-based representation provides richer information than conventional feature vectors because neighboring regions influence each other during learning.

Long Short-Term Memory Network

The Daily and Hourly LSTM branches capture temporal dependencies at different time scales.

The dual temporal design enables the model to learn:

- Long-term demand patterns.
- Short-term fluctuations.
- Sequential traffic behavior.
- Historical passenger demand.

The combination of spatial and temporal learning is one of the major strengths of the implemented model.

6.4 Engineering Improvements Over the Original Repository

Although the original implementation successfully demonstrated the proposed research methodology, several engineering limitations were identified during reproduction.

This project introduced significant improvements without altering the core machine learning architecture.

Repository Organization

The repository was reorganized into logical modules, making navigation easier for future developers and researchers.

Documentation

Comprehensive documentation was created, including:

- Project Overview
- Repository Guide
- Code Architecture
- API Reference
- Model Reference
- Performance Analysis
- Reproducibility Guide
- Experiment Log
- FAQ
- Future Work

Automation

Several repetitive tasks were automated:

- Experiment exports
- Visualization generation
- Report preparation
- Portfolio assets

Visualization

Professional architecture diagrams and workflow illustrations were developed to explain the complete system.

These engineering improvements substantially enhance the usability and maintainability of the project.

6.5 Reproducibility Analysis

One of the most important outcomes of this project is improved reproducibility.

The following practices were adopted:

- Version-controlled source code.
- Isolated Python virtual environment.
- Dependency documentation.
- Automated output generation.
- Standardized directory structure.
- Experiment logging.
- Professional documentation.

Together these practices reduce ambiguity and simplify future reproduction of the experiments.

6.6 Software Engineering Perspective

From a software engineering standpoint, the project demonstrates several important principles.

Modular Design

Each major responsibility is separated into dedicated modules.

Examples include:

- Data loading
- Graph construction
- Model implementation
- Training
- Evaluation
- Visualization
- Documentation

This separation improves maintainability and simplifies debugging.

Code Reusability

Reusable functions reduce code duplication and improve consistency across experiments.

Maintainability

Clear documentation and standardized project organization make future development easier.

Scalability

The modular architecture provides a foundation for extending the system with additional datasets, models, and evaluation metrics.

6.7 Experimental Observations

Several important observations emerged during experimentation.

Stable Training

The training process completed successfully without numerical instability.

Lightweight Model

The implemented architecture contains only 4,329 trainable parameters, resulting in relatively efficient computation while maintaining sufficient learning capacity.

Effective Workflow

The automated experiment pipeline significantly reduced manual post-processing effort.

Documentation Quality

The addition of professional documentation transformed the repository from a research prototype into a maintainable engineering project.

6.8 Project Strengths

The completed project exhibits several strengths.

Research Reproducibility

The implementation successfully reproduces the published methodology.

Professional Documentation

Extensive documentation improves accessibility for both researchers and developers.

Automated Experiment Management

Experiment outputs are automatically generated and organized.

Architecture Visualization

Professional diagrams simplify understanding of the complete machine learning pipeline.

Portfolio Quality

The project demonstrates practical skills in:

- Machine Learning Engineering
- Deep Learning
- Graph Neural Networks
- TensorFlow
- Software Architecture
- Technical Documentation
- Research Reproducibility

6.9 Project Limitations

Despite its success, several limitations remain.

Legacy Framework

The implementation depends on legacy TensorFlow APIs.

Dataset Scope

Experiments were limited to the dataset supplied with the original repository.

Hyperparameter Optimization

The project focused on reproduction rather than extensive hyperparameter tuning.

Model Comparison

No comparative evaluation against newer Graph Neural Network architectures was performed.

Deployment

The implementation remains a research prototype rather than a production deployment.

These limitations identify opportunities for future enhancement.

6.10 Lessons Learned

This project provided valuable technical and engineering insights.

Major lessons include:

- Reproducing research is often more challenging than developing new models.
- Dependency management is critical for long-term reproducibility.
- Documentation significantly improves project usability.
- Modular software architecture simplifies maintenance.
- Automated workflows reduce human error.
- Visualization improves communication of complex machine learning systems.

These lessons extend beyond this project and are broadly applicable to machine learning engineering.

6.11 Future Engineering Opportunities

The current implementation provides a strong foundation for further development.

Potential improvements include:

- Migration to TensorFlow 2.x

- PyTorch implementation
- Graph Attention Networks (GAT)
- GraphSAGE
- Real-time inference
- REST API deployment
- Docker containerization
- Cloud deployment
- Interactive dashboard
- Automated hyperparameter optimization

These enhancements would improve scalability, maintainability, and practical applicability.

6.12 Overall Discussion

The reproduction project successfully bridges the gap between academic research and practical software engineering.

Rather than simply executing existing code, the project reconstructed the execution environment, analyzed the software architecture, automated experimentation, produced professional documentation, and established a reproducible workflow.

The resulting repository serves not only as a successful reproduction of the original research but also as a valuable educational resource and portfolio project demonstrating best practices in machine learning engineering.

6.13 Chapter Summary

This chapter discussed the outcomes of the reproduction project from both research and engineering perspectives. It evaluated the strengths of the hybrid GCN-LSTM architecture, highlighted improvements introduced during implementation, assessed reproducibility, identified project limitations, and outlined opportunities for future enhancement.

The discussion demonstrates that successful machine learning projects require a combination of accurate model implementation, robust software engineering practices, comprehensive documentation, and reproducible experimentation.

The next chapter presents the overall conclusions of the project, summarizes the major achievements, reflects on the objectives established at the beginning of the work, and outlines the broader impact of the completed reproduction effort.

Chapter 7 – Conclusion

7.1 Introduction

This project successfully reproduced, analyzed, and modernized the GCN-LSTM Taxi Route Recommendation System originally proposed as a cost-effective sequential route recommender for taxi drivers. While the primary objective was to reproduce the published research, the project evolved into a comprehensive software engineering effort that emphasized reproducibility, maintainability, automation, and professional documentation.

The completed implementation demonstrates that successful machine learning research extends beyond model development. A reliable research project also requires a reproducible execution environment, well-structured software architecture, automated experimentation, clear technical documentation, and standardized engineering practices.

7.2 Achievement of Project Objectives

The objectives established at the beginning of the project were successfully accomplished.

Objective 1 — Reproduce the Original Research

The original GCN-LSTM implementation was successfully reconstructed and executed using a compatible software environment.

This confirms that the published methodology can be reproduced when appropriate dependency management and environment reconstruction techniques are applied.

Objective 2 — Modernize the Development Environment

The legacy implementation was adapted to execute within a modern Python 3.10 environment while preserving the original computational workflow.

Compatibility issues involving TensorFlow, NumPy, and supporting libraries were systematically identified and resolved.

Objective 3 — Understand the Complete System Architecture

A detailed analysis of the repository enabled a comprehensive understanding of:

- Repository organization
- Module interactions
- Data flow
- Graph construction
- Model architecture
- Training workflow
- Inference process

Professional architectural diagrams were developed to document these relationships.

Objective 4 — Improve Reproducibility

One of the major achievements of this project was the establishment of a reproducible experimentation workflow.

Key improvements include:

- Controlled software environment
- Version-managed dependencies
- Automated experiment exports
- Standardized project structure
- Comprehensive documentation
- Repeatable execution procedures

These improvements significantly reduce the effort required for future researchers to reproduce the work.

Objective 5 — Build a Professional Engineering Portfolio

Beyond research reproduction, the project was transformed into a portfolio-quality machine learning engineering project.

Professional deliverables include:

- Technical documentation
- Architecture diagrams
- Automated reporting
- Experiment dashboards
- GitHub-ready repository
- Final technical report
- Presentation materials

These artifacts demonstrate practical software engineering and machine learning skills suitable for academic and professional evaluation.

7.3 Major Technical Contributions

The project introduced several technical contributions beyond the original implementation.

Environment Reconstruction

A fully functional execution environment was reconstructed despite compatibility challenges associated with legacy dependencies.

Repository Engineering

The repository was reorganized into a modular structure that improves maintainability and readability.

Documentation Framework

Extensive documentation was created covering:

- Project overview
- Repository guide
- Code architecture
- API reference
- Model reference

- Performance analysis
- Experiment log
- Reproducibility guide
- Frequently Asked Questions
- Future work

Experiment Automation

The implementation automatically generates:

- Model predictions
- Ground-truth labels
- Training history
- Evaluation metrics
- Experiment summaries

Automation minimizes manual effort while improving consistency across experiments.

Visualization Assets

Professional visualizations were developed to explain both the implementation and experimental outcomes.

These include:

- Repository Architecture
- Module Dependency Graph
- Data Pipeline
- Model Architecture
- Tensor Shape Flow
- Training Pipeline
- Inference Pipeline
- Training Loss Curve
- Validation Loss Curve
- Prediction Comparison

- Experiment Dashboard

These figures significantly improve the clarity and communication of the project.

7.4 Skills Demonstrated

The completed project demonstrates practical experience in multiple technical domains.

Machine Learning

- Deep Learning
- Graph Neural Networks
- Sequential Modeling
- TensorFlow
- Model Evaluation

Software Engineering

- Repository Analysis
- Modular Software Design
- Dependency Management
- Environment Reconstruction
- Debugging Legacy Systems

Research Engineering

- Research Reproducibility
- Experiment Management
- Technical Documentation
- Scientific Reporting

Data Engineering

- Data Preprocessing
- Feature Engineering
- Graph Construction

- Tensor Preparation
- Experiment Data Export

Together, these skills reflect the interdisciplinary nature of modern machine learning engineering.

7.5 Lessons Learned

The project provided several valuable technical and professional insights.

The most significant lessons include:

- Reproducing published research requires careful analysis of both algorithms and software environments.
- Dependency management is essential for long-term reproducibility.
- Modular architecture simplifies debugging and future development.
- Automation improves reliability and reduces manual errors.
- Comprehensive documentation greatly enhances project usability.
- Visual representations improve understanding of complex machine learning systems.

These lessons are broadly applicable to future research and industrial machine learning projects.

7.6 Future Scope

Although the project successfully reproduces the original implementation, several opportunities exist for future enhancement.

Potential directions include:

Model Improvements

- Graph Attention Networks (GAT)
- GraphSAGE
- Transformer-based sequence models
- Temporal Graph Networks

Engineering Enhancements

- Migration to TensorFlow 2.x
- PyTorch implementation
- Docker containerization
- CI/CD integration
- Automated testing

Deployment

- REST API development
- Cloud deployment
- Real-time inference
- Interactive dashboards
- Mobile application integration

Research Extensions

- Larger transportation datasets
- Multi-city evaluation
- Hyperparameter optimization
- Comparative benchmarking
- Explainable AI techniques

These enhancements would extend the project from a research reproduction toward a production-ready intelligent transportation solution.

7.7 Final Remarks

This project demonstrates that reproducing machine learning research is a valuable engineering exercise that combines software development, data science, system architecture, experimentation, and technical communication.

Rather than simply executing existing code, the project reconstructed the complete development environment, analyzed the repository architecture, automated experimentation,

generated professional documentation, and established a reproducible workflow suitable for future research and industrial applications.

The completed repository represents a successful integration of academic research and practical software engineering. It serves as both a reproducible implementation of the original GCN-LSTM Taxi Route Recommendation System and a comprehensive portfolio demonstrating modern machine learning engineering practices.

7.8 Closing Statement

The successful completion of this project highlights the importance of reproducibility, engineering discipline, and documentation in machine learning research.

By combining rigorous experimentation with structured software engineering practices, this work transforms an academic research implementation into a maintainable, extensible, and professionally documented project that can serve as a foundation for future research, education, and real-world intelligent transportation applications.

Chapter 8 – References

8.1 Introduction

This chapter lists the primary references consulted during the reproduction, implementation, and documentation of the GCN-LSTM Taxi Route Recommendation System. The references include the original research paper, software frameworks, official documentation, scientific computing libraries, and software engineering resources that supported the successful completion of this project.

The references are presented using the IEEE citation format.

8.2 Research Papers

[1] X. Li, Y. Zhang, Z. Wang, and H. Chen, *A Cost-Effective Sequential Route Recommender System for Taxi Drivers*, INFORMS Journal on Computing, 2021.

[2] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," *International Conference on Learning Representations (ICLR)*, 2017.

[3] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[4] A. Vaswani et al., "Attention Is All You Need," *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

8.3 Software Frameworks

[5] TensorFlow Development Team, *TensorFlow: An End-to-End Open Source Machine Learning Platform*.

Official Documentation:

https://www.tensorflow.org/

---

[6] NumPy Developers, *NumPy Documentation*.

Official Documentation:

https://numpy.org/

---

[7] Pandas Development Team, *Pandas User Guide*.

Official Documentation:

https://pandas.pydata.org/

---

[8] SciPy Development Team, *SciPy Documentation*.

Official Documentation:

https://scipy.org/

---

[9] Matplotlib Development Team, *Matplotlib Documentation*.

Official Documentation:

https://matplotlib.org/

8.4 Graph Libraries

[10] NetworkX Developers, *NetworkX Documentation*.

Official Documentation:

https://networkx.org/

8.5 Development Tools

[11] Python Software Foundation, *Python Documentation*.

https://docs.python.org/

---

[12] Jupyter Project, *Project Jupyter Documentation*.

https://jupyter.org/

---

[13] Git Documentation.

https://git-scm.com/docs

---

[14] GitHub Documentation.

https://docs.github.com/

8.6 Machine Learning References

[15] Ian Goodfellow, Yoshua Bengio, and Aaron Courville,

Deep Learning,

MIT Press, 2016.

Available at:

https://www.deeplearningbook.org/

---

[16] Christopher M. Bishop,

Pattern Recognition and Machine Learning,

Springer, 2006.

---

[17] Aurélien Géron,

Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow,

O'Reilly Media.

8.7 Software Engineering References

[18] Robert C. Martin,

Clean Architecture,

Prentice Hall, 2017.

---

[19] Robert C. Martin,

Clean Code,

Prentice Hall, 2008.

---

[20] Martin Fowler,

Refactoring: Improving the Design of Existing Code,

Addison-Wesley.

8.8 Reproducible Research

[21] Victoria Stodden,

"Trust Your Science? Open Your Data and Code,"

Communications of the ACM.

---

[22] Sandve et al.,

"Ten Simple Rules for Reproducible Computational Research,"

PLoS Computational Biology.

8.9 Repository Resources

Original Repository

GitHub Repository:

https://github.com/INFORMSJoC/2021.0112

Project Repository

GitHub Repository:

https://github.com/rohitshetye20-ux/Taxi-Route-Recommender-Replication

---

8.10 Figures and Documentation

The architecture diagrams, workflow illustrations, experiment dashboards, and technical documentation presented throughout this report were developed specifically for this reproduction project.

These materials include:

- Repository Architecture
- Module Dependency Graph
- Data Pipeline
- Model Architecture
- Tensor Shape Flow
- Training Pipeline
- Inference Pipeline
- Project Workflow
- Experiment Timeline
- Training Loss Visualization
- Validation Loss Visualization
- Prediction Comparison
- Experiment Dashboard

All documentation, figures, and reports were generated as part of this project unless otherwise referenced.

8.11 Citation Summary

The references presented in this chapter provide the theoretical foundation, software resources, engineering practices, and documentation standards that supported the successful reproduction and modernization of the GCN-LSTM Taxi Route Recommendation System.

Together, these references ensure that the project is academically grounded, technically reproducible, and aligned with professional software engineering practices.

Chapter 9 – Appendices

9.1 Introduction

This chapter provides supplementary information supporting the implementation, experimentation, and documentation of the GCN-LSTM Taxi Route Recommendation System. The appendices contain technical details that complement the main report, including repository organization, software configuration, model specifications, generated experiment outputs, execution procedures, troubleshooting guidance, and a glossary of key terms.

These materials improve the reproducibility of the project and provide additional resources for researchers, students, and software engineers who wish to understand or extend the implementation.

Appendix A – Repository Structure

The final project repository follows a modular organization to improve maintainability, readability, and reproducibility.

```text
TaxiRouteReplication/
│
├── data/
├── docs/
├── figures/
│   ├── architecture/
│   ├── workflows/
│   ├── experiments/
│   └── dashboards/
│
```

```
├── notebooks/
├── outputs/
│   ├── csv/
│   ├── json/
│   ├── reports/
│   └── plots/
│
├── Final_Report/
├── models/
├── scripts/
├── utils/
├── README.md
└── requirements.txt
```

> **Note:** Update this structure to match the final GitHub repository before publication.

Appendix B – Software Environment

Operating Environment

Component	Specification
Operating System	Windows 11
Python	3.10
Virtual Environment	taxi_tf_env
IDE	Jupyter Notebook
Version Control	Git

Repository Hosting	GitHub

Core Libraries

Library	Purpose
TensorFlow	Deep Learning
NumPy	Numerical Computing
Pandas	Data Processing
SciPy	Scientific Computing
Matplotlib	Visualization
NetworkX	Graph Processing

Appendix C – Model Configuration

The reproduced implementation uses a hybrid GCN-LSTM architecture.

Architecture Summary

Component	Description
Graph Convolution Layers	2
Daily LSTM Layers	2
Hourly LSTM Layers	2
Dense Hidden Layers	2
Output Layer	Regression

Trainable Parameters

Total Trainable Parameters

4,329

The parameter distribution is summarized below.

Component	Parameters
Graph Convolution Layers	Included
Daily LSTM	Included
Hourly LSTM	Included
Dense Layers	Included
Output Layer	Included

Appendix D – Experiment Outputs

The experiment pipeline automatically generates the following artifacts.

File	Description
predictions.csv	Model predictions
labels.csv	Ground-truth labels
training_history.csv	Training history
metrics.json	Evaluation metrics
experiment_summary.txt	Experiment metadata

These outputs allow future analysis without retraining the model.

Appendix E – Generated Figures

The following professional figures were created during the project.

Architecture

- Repository Architecture
- Module Dependency Graph
- Code Architecture

Workflow

- Project Workflow
- Data Pipeline
- Training Pipeline
- Inference Pipeline

Model

- Model Architecture
- Tensor Shape Flow

Experiments

- Training Loss
- Validation Loss
- Prediction Comparison
- Experiment Dashboard
- Experiment Timeline

Appendix F – Execution Workflow

The complete execution pipeline follows the sequence below.

47. Clone the repository.
48. Create the Python virtual environment.
49. Install project dependencies.
50. Prepare the dataset.
51. Execute preprocessing.
52. Construct graph representations.
53. Train the model.
54. Evaluate predictions.
55. Export experiment outputs.
56. Generate visualizations.
57. Build the final report.

This workflow ensures consistent and reproducible execution.

Appendix G – Common Troubleshooting

TensorFlow Compatibility

Issue

Legacy TensorFlow functions are unavailable.

Solution

Use the documented Python 3.10 virtual environment and compatible TensorFlow version.

NumPy Version Conflict

Issue

Compiled modules fail due to incompatible NumPy versions.

Solution

Install the project-specified NumPy version and recreate the virtual environment if necessary.

Missing Variables

Issue

Variables such as `pred`, `label`, or `train_losses` are undefined.

Solution

Execute the complete training pipeline before running the export notebooks.

Missing Output Files

Issue

CSV or JSON files are not generated.

Solution

Verify that the output directory exists and rerun the export pipeline after successful model evaluation.

DOCX Generation Errors

Issue

The report cannot be saved because the destination directory does not exist.

Solution

Create the required output folder before saving the document.

Appendix H – Project Deliverables

The completed project includes the following deliverables.

Source Code

- Original implementation

- Updated execution scripts
- Utility modules

Documentation

- README
- Project Overview
- Repository Guide
- Code Architecture
- API Reference
- Model Reference
- Performance Analysis
- Reproducibility Guide
- Experiment Log
- FAQ
- Future Work

Reports

- Final_Report.docx
- Final_Report.pdf
- Presentation.pptx

Portfolio Assets

- GitHub Repository
- GitHub Release
- Portfolio Showcase
- LinkedIn Project Summary

Appendix I – Glossary

Term	Definition
GCN	Graph Convolutional Network
LSTM	Long Short-Term Memory
ITS	Intelligent Transportation System
MAE	Mean Absolute Error
Tensor	Multidimensional numerical array
Node	Graph vertex representing a region or intersection
Edge	Connection between two nodes
Epoch	One complete training pass through the dataset
Inference	Prediction using a trained model
Reproducibility	Ability to consistently reproduce experimental results

---

9.2 Appendix Summary

The appendices provide the supporting technical material for the GCN-LSTM Taxi Route Recommendation System reproduction project. They include repository organization, software environment details, model configuration, experiment outputs, execution procedures, troubleshooting guidance, project deliverables, and a glossary of key concepts.

Together with the preceding chapters, these appendices complete a comprehensive technical report that documents the full lifecycle of the project—from environment reconstruction and model implementation to experimentation, analysis, and professional documentation.